Non Attacking Queens

AlgoExpert · Very Hard · Recursion

The problem

- Number of ways to place n queens on an n x n board with no two attacking.
- Backtracking with column/diagonal bitmasks.

Intuition

- Place one queen per row; track used columns and both diagonals for constant-time checks.

Brute force vs optimal

- Try all placements → $C(n^2, n)$
- Backtracking row by row with sets → $O(n!)$
- Bitmasks make checks $O(1)$

Approach

- 1. Recurse row by row
- 2. Try each free column
- 3. Mark column and diagonals
- 4. Count full placements

Worked example

one queen per row

bitmasks: used columns, ↘ diagonals, ↙ diagonals

Place row by row with bitmasks

Worked example

```
free = ~(cols | d1 | d2) & full

take lowest bit, recurse with shifted diagonals
```

Iterate over free columns

Worked example

`n=4 → 2 solutions`

`n=8 → 92`

Results

Java solution

```java
public class NonAttackingQueens {
    public static int nonAttackingQueens(int n) { return place(n, 0, 0, 0, 0); }

    private static int place(int n, int row, int cols, int d1, int d2) {
        if (row == n) return 1;
        int count = 0;
        int free = ~(cols | d1 | d2) & ((1 << n) - 1);
        while (free != 0) {
            int bit = free & -free;
            free -= bit;
            count += place(n, row + 1, cols | bit, (d1 | bit) << 1, (d2 | bit) >> 1);
        }
        return count;
    }
}
```

Complexity

- Time: O(n!)
- Space: O(n)

Edge cases & pitfalls

- Shift diagonals each row
- Mask to n bits
- N = 2, 3 → 0

Interview tips & follow-ups

- Mention symmetry pruning

Recap

- Place one queen per row; track used columns and both diagonals for constant-time checks.
- Complexity: O(n!)